

Copying converting and creating data

Лекция и упражнения по теме

CONSTRUCT выдает в качестве результата запроса тройки. Можно взять тройки непосредственно из источника данных без изменения, или можно извлечь значения, и использовать эти значения, чтобы создать новые тройки. Это позволяет копировать, создавать, конвертировать данные RDF, и это позволяет идентифицировать данные, которые не соответствуют определенным бизнес-правилам.

ASK спрашивает процессор запросов, описывает ли заданный шаблон графа набор троек в конкретном наборе данных или нет, и процессор возвращает логическое **истина** или **ложь**. Это отличная возможность для выражения бизнес-правил об условиях, которые должны быть или не должны быть верны в ваших данных. Можно использовать наборы из этих правил для автоматизации контроля качества в конвейере обработки данных.

DESCRIBE просит тройки, которые описывают конкретный ресурс. Спецификация SPARQL оставляет это решение процессору запросов, какие тройки выдать в качестве описания имени ресурса. Это привело к несогласованности реализаций запросов DESCRIBE, так что это форма запроса не очень популярна.

Пример копирования триплетов(1)

PREFIX dm: <DEMO>

CONSTRUCT {?s ?p ?o}

FROM <<http://dbpedia.org/sparql>>

WHERE

{ ?s ?p ?o }LIMIT 100

Копирование триплетов с пустым шаблоном CONSTRUCT (2)

PREFIX dm: <DEMO>

CONSTRUCT

WHERE

{ ?s ?p ?o }

Копирование триплетов из именованного графа (3)

```
CONSTRUCT {?s ?p ?o.}  
FROM NAMED <http://dbpedia.org/resource/classes#>  
where {GRAPH <http://dbpedia.org/resource/classes#>  
{?s ?p ?o}} LIMIT 100
```

Копирование триплетов из нескольких источников

(4)

```
PREFIX cat: <http://dbpedia.org/resource/Category:>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX gp: <http://wifo5-04.informatik.uni-mannheim.de/gutendata/resource/people/>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
CONSTRUCT
{
  <http://dbpedia.org/resource/Joseph_Hocking> ?dbpProperty ?dbpValue .
  gp:Hocking_Joseph ?gutenProperty ?gutenValue .
}
WHERE
{
  SERVICE <http://DBpedia.org/sparql>
  {
    <http://dbpedia.org/resource/Joseph_Hocking> ?dbpProperty ?dbpValue .
  }
  SERVICE <http://wifo5-04.informatik.uni-mannheim.de/gutendata/sparql>
  {
    gp:Hocking_Joseph ?gutenProperty ?gutenValue .
  }
}
```

Создание новых данных с пустым шаблоном

WHERE

```
PREFIX dc: <http://purl.org/dc/elements/1.1/>
```

```
PREFIX dm: <DEMO>
```

```
CONSTRUCT
```

```
{
```

```
  <http://dbpedia.org/resource/Novosibirsk>  
  dc:title "NSO" .
```

```
}
```

```
WHERE
```

```
{}
```

CONSTRUCT для создания новых данных

PREFIX dbpedia: <http://dbpedia.org/resource/>

PREFIX dbo: <http://dbpedia.org/ontology/>

PREFIX dc: <http://purl.org/dc/elements/1.1/>

PREFIX foaf: <http://xmlns.com/foaf/0.1/>

PREFIX mo: <http://purl.org/ontology/mo/>

PREFIX dm:<DEMO>

CONSTRUCT {?work dc:description "About Love"}

WHERE {

 ?work rdf:type [dbo:Song](#);

[dbo:artist](#) <http://dbpedia.org/resource/The_Beatles>.

FILTER(REGEX(?work, "love", "i"))

}

Преобразование данных

PREFIX cat: <<http://dbpedia.org/resource/Category:>>

PREFIX foaf: <<http://xmlns.com/foaf/0.1/>>

PREFIX gp: <<http://wifo5-04.informatik.uni-mannheim.de/gutendata/resource/people/>>

PREFIX owl: <<http://www.w3.org/2002/07/owl#>>

PREFIX rdf: <<http://www.w3.org/1999/02/22-rdf-syntax-ns#>>

PREFIX rdfs: <<http://www.w3.org/2000/01/rdf-schema#>>

PREFIX skos: <<http://www.w3.org/2004/02/skos/core#>>

PREFIX d: <<http://learningsparql.com/ns/data#>>

CONSTRUCT

{

 d:HockingJoseph ?dbpProperty ?dbpValue ;
 ?gutenProperty ?gutenValue .

}

WHERE

{

 SERVICE <<http://DBpedia.org/sparql>>

 {
 <http://dbpedia.org/resource/Joseph_Hocking> ?dbpProperty ?dbpValue .
 }

 SERVICE <<http://wifo5-04.informatik.uni-mannheim.de/gutendata/sparql>>

 {
 gp:Hocking_Joseph ?gutenProperty ?gutenValue .
 }

}

ASK для поиска плохих данных

При разработке реляционных баз данных, XML и других областях информационных технологий, схема представляет собой набор правил о структурах данных и типах для обеспечения качества данных и более эффективных систем.

Если одна из этих схем говорит, что значение переменной *количество* должно быть целым, вы знаете, что оно никогда не будет равным 3,5 или "привет".

Таким образом, разработчики приложения для обработки данных, не обязаны беспокоиться о странных данных, которые сломают обработку: если программа вычитает 1 из *количество* и переменная *количество* оказалась равной "привет", это может привести к беде.

Если данные соответствуют правильной схеме, разработчик, использующий данные, не обязан писать код, чтобы учесть такую возможность.

ASK для поиска плохих данных

- RDF -приложения используют иной подход.
- RDF Schema и онтологии OWL добавляют дополнительные метаданные. Например, когда мы знаем, что ресурс *d: id432* является членом класса *d: product3973*, который имеет *rdfs:label* "клубника" и является подклассом класса с *rdfs:label* "фрукт", мы знаем, что *d: product3973* также является членом класса "фрукт".
- Это здорово, но что, если вы хотите определить правила для троек и проверить, соответствует ли им набор данных так, что приложению не придется беспокоиться о неожиданных значениях данных, нарушая логику? OWL обеспечивает несколько способов сделать это, но они могут получиться довольно сложными, и вы будете нуждаться в процессоре OWL.
- Использование SPARQL для того, чтобы определить такие ограничения становятся все более популярными, как из-за своей простоты так и более широкого спектра программного обеспечения (то есть, все процессоры SPARQL), которые позволяют вам реализовать эти правила.
- В качестве бонуса, те же самые методы позволяют определить бизнес-правила, которые полностью выходят за рамки SQL в развитии реляционной базы данных. Они также за пределами традиционных схем XML.

Польза от запросов ASK

@prefix dm: <<http://learningsparql.com/ns/demo#>> .

@prefix d: <<http://learningsparql.com/ns/data#>> .

d:item432 dm:cost 8.50 ;

dm:amount 14 ;

dm:approval d:emp079 ;

dm:location <<http://dbpedia.org/resource/Boston>> .

d:item201 dm:cost 9.25 ;

dm:amount 12 ;

dm:approval d:emp092 ;

dm:location <<http://dbpedia.org/resource/Ghent>> .

d:item857 dm:cost 12 ;

dm:amount 10 ;

dm:location <<http://dbpedia.org/resource/Montreal>> .

d:item693 dm:cost 10.25 ;

dm:amount 1.5 ;

dm:location "Heidelberg" .

d:item126 dm:cost 5.05 ;

dm:amount 4 ;

dm:location <<http://dbpedia.org/resource/Lisbon>> .

d:emp092 dm:jobGrade 1 .

d:emp041 dm:jobGrade 3 .

d:emp079 dm:jobGrade 5 .

Правила, которые хочется применять к данным, и их нарушения

- Все значения dm:location должны быть URI, потому что хочется связать эти данные с другими соответствующими данными. Пункт d: item693 имеет значение dm:location "Гейдельберг", который является строкой, а не URI.
- Все значения dm:amount должны быть целыми числами. Выше, d: item693 имеет значение dm:amount 1,5, которое не хотелось бы отправить приложению.
- Скорее бизнес-правило, чем правило проверки данных: значение dm:approval считается необязательным, если общая стоимость покупки меньше или равна 100. Если она больше 100, покупка должна быть одобрена работником с классом работы большей, чем 4.
- Приобретение 14 предметов d: item432 по цене 8.50 каждый стоит больше, чем 100, но это одобрено кем-то с классом работы 5, так что все нормально. d: item126 не имеет одобрение в списке, но при общей стоимости 20.20, он не нуждается в утверждении.
- Однако, d: item201 стоит более 100, и при утверждении сотрудник имеет класс работы 1, а d: item857 также стоит более 100 и не имеет одобрения вообще, поэтому надо поймать их.

Имеется ли в наборе данных нарушение
правила 1?

PREFIX dm:

<http://learningsparql.com/ns/demo#>

ASK WHERE

{

?s dm:location ?city .

FILTER (!(isURI(?city)))

}

Имеется ли в наборе данных нарушение
правила 2?

PREFIX dm: <http://learningsparql.com/ns/demo#>

PREFIX xsd:

<http://www.w3.org/2001/XMLSchema#>

ASK WHERE

{

?item dm:amount ?amount .

FILTER ((datatype(?amount)) != xsd:integer)

}

Имеется ли в наборе данных нарушение правила 3?

PREFIX dm: <http://learningsparql.com/ns/demo#>

ASK WHERE

{

 ?item dm:cost ?cost ;
 dm:amount ?amount .

OPTIONAL

{

 ?item dm:approval ?approvingEmployee .
 ?approvingEmployee dm:jobGrade ?grade .

}

BIND (?cost * ?amount AS ?totalCost) .

FILTER ((?totalCost > 100) &&
 ((!bound(?grade)) || (?grade < 5))))

}

Запрос, показывающий, где именно ошибка 1

```
PREFIX dm: <http://learningsparql.com/ns/demo#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
CONSTRUCT
{
  ?s dm:problem dm:prob29 .
  dm:prob29 rdfs:label "Location value must be a URI." .
}
WHERE
{
  ?s dm:location ?city .
  FILTER (!(isURI(?city)))
}
```

Результат запроса, указывающий место ошибки

@prefix rdfs:

<<http://www.w3.org/2000/01/rdfschema#>> .

@prefix d:

<<http://learningsparql.com/ns/data#>> .

@prefix dm:

<<http://learningsparql.com/ns/demo#>> .

dm:prob29

rdfs:label "Location value must be a URI." .

[d:item693](#)

dm:problem dm:prob29 .